Context-aware vulnerability classification with knowledge graphs and security intelligence

Krit Meesiri, Panod Manodee, Apichat Laohawan, Sujitta Sonna
Sirindhorn International Institute of Technology, Thammasat University, Thailand
6622780037@g.siit.tu.ac.th, 6622782033@g.siit.tu.ac.th, 6622770491@g.siit.tu.ac.th, 6622770640@g.siit.tu.ac.th

***Abstract*—Modern Security Operations Centers (SOCs) rely on heterogeneous monitoring platforms such as Wazuh, Splunk, Microsoft Sentinel, and AMiner, whose outputs differ in format and abstraction level, requiring analysts to manually correlate security findings across tools. Moreover, standardized vulnerability representations such as CVE, CWE, and CVSS describe intrinsic vulnerability characteristics but provide limited information about the operational context in which vulnerabilities are deployed. This paper presents a context-aware vulnerability classification and risk prioritization system that addresses these challenges through a unified security analysis pipeline. External threat intelligence, including: cvelist, NVD database, MITRE ATT&CK, MITRE CWE, and MITRE CAPEC, along with computer assets and network topology data collected from inventory scanners or other tools are used to create a Cyber Threat Intelligence (CTI) knowledge and Computer Assets (CA) knowledge graph respectively. Then the two knowledge graphs are integrated and linked together to create the Unified Knowledge Graph. Vulnerability alerts from security tools like Wazuh are used to query the knowledge graph to calculate the blast radius of the vulnerability impact. The security requirements of the nodes within the affected zone are used to calculate the Environmental Vulnerability Score (EVS) and classify the impact of the alert. the proposed approach enables the networking and asset contexts to be integrated with the CVSS score from threat intelligence to provide a context-aware vulnerability score and classification.**

I. Introduction

Modern Security Operations Centers (SOCs) rely on heterogeneous monitoring platforms such as Wazuh, Splunk, Microsoft Sentinel, and AMiner, each producing findings in different formats and levels of abstraction. This forces analysts to manually correlate information across tools to understand a single incident. Furthermore, standardized vulnerability representations such as CVE, CWE, and CVSS describe a vulnerability's intrinsic characteristics but do not reflect the environment in which it is deployed—factors such as asset criticality, network exposure, and observed threat activity. As a result, the same vulnerability may pose substantially different levels of risk depending on its deployment context, motivating the need for a context-aware approach to vulnerability classification and prioritization.

Prior work has addressed parts of this problem in isolation. Knowledge graph-based approaches [1], [4], [8], [9], [10] model relationships among vulnerabilities, assets, and threat intelligence, but typically rely on manually curated or report-derived intelligence rather than heterogeneous operational security logs and lack a dedicated learning-based extraction stage. Classification-based approaches such as V2W-BERT [6] and DeKeDVer [2] map vulnerability descriptions to categories such as CWE, but operate on the description in isolation, producing context-free classifications that do not reflect the deployment environment. Consequently, existing approaches do not provide a unified pipeline that combines entity-level extraction from heterogeneous security logs, machine learning-based classification, and graph-based context modeling to produce an environment-aware risk assessment rather than a static vulnerability label.

This paper addresses this gap by proposing a context-aware pipeline for vulnerability classification and prioritization. External threat intelligence and local computing asset data are collected and processed to construct two knowledge graphs (KGs): a Cyber Threat Intelligence (CTI) knowledge graph and a Computer Assets (CA) knowledge graph. These are then integrated into a unified Vulnerability Knowledge Graph (VKG) that captures the relationships between known threats and the organization's assets. Security alerts generated by monitoring tools such as Wazuh, Suricata, and Microsoft Sentinel are used to query the VKG and identify the affected assets. The base scores of the reported vulnerabilities are then recalculated using weights derived from the affected assets, yielding an Environmental Vulnerability Score (EVS) that reflects the organization's specific operational context. The main contributions of this paper is as follows:

- **A Domain-specific Threat Intelligence-assisted Knowledge Graph Representation** We implement a knowledge graph that represents the relationships and correlations between a domain-specific computer assets knowledge graph and a cybersecurity threat intelligence.
- **Context-Aware Classification Model** We design a model to compute the environmental vulnerability score and classify the vulnerability according to the environmental context of the system.
- **Cybersecurity tool integration** We integrate commercial cybersecurity tools in our design as the trigger to query the VKG to calculate the EVS
- **Context-Aware Threat Report** We use a Large Language Model (LLM) to transform the calculated vulnerability assessment results into structured threat reports, including incident summaries, affected assets, vulnerability severity, EVS results, and recommended mitigation measures.

II. Related Work

Nagaty et al. [1] developed a knowledge graph unification pipeline that constructs and integrates threat intelligence and computer asset knowledge graphs. Their work proposes a five-phase framework:

1) **Data collection.** Two data streams are gathered: threat data from public cyber threat intelligence sources, and computer asset data from inventory tools and internal databases.
2) **Graph construction.** Two subgraphs are built. In the Cyber Threat Graph, threat intelligence entities are represented as nodes and linked through typed relationships. In the Computer Asset Graph, Common Platform Enumeration (CPE) identifiers are linked to the corresponding vulnerabilities in the Cyber Threat Graph.
3) **Graph integration.** The two graphs are merged through entity alignment and entity resolution. The authors describe this step conceptually as an NLP-based process that uses named entity recognition (NER) and embeddings, with aligned node pairs represented as a product-combination graph. In practice, however, the integration they implemented relied on string- and text-similarity matching of entity names and descriptions between the CWE and CAPEC graphs.
4) **GNN enhancement.** The authors propose using graph neural networks (GNNs) to predict missing links, such as hidden attack paths or previously unidentified threat–asset relationships.
5) **Insight generation.** The authors propose using Neo4j for monitoring and visualization.

This work is closely related to our research, as it addresses many of the challenges central to our objectives. However, it concludes at the graph unification stage, and its link prediction, monitoring, and visualization components remain proposals rather than implementations. Our work extends this line of research by adopting their knowledge graph construction pipeline and building a context-aware vulnerability classification model on top of it.

Automated vulnerability classification has been widely studied to accelerate vulnerability triage and CVE-to-CWE mapping. Dong et al. In [3] fine-tuned BERT to jointly classify vulnerability categories and hazard levels from crawled vulnerability reports, achieving only 61.99–65.02% accuracy. These results highlight the difficulty of text-based classification when the label space becomes large and highly imbalanced. Das et al. In [8] addressed this limitation by formulating CVE-to-CWE assignment as a link-prediction problem rather than direct classification, achieving 89–98% accuracy on frequent CWE classes and 48–76% on rare classes under a temporal train/test split. Despite these improvements, both approaches rely primarily on vulnerability descriptions and the CWE hierarchy. Neither explicitly represents the asset affected by a vulnerability or the operational environment in which it is deployed.

Knowledge graph approaches have been explored to integrate and reason over cybersecurity knowledge from multiple sources. Rahman et al. In [4] developed an automated pipeline for periodically ingesting CVE, CWE, CAPEC, and ATT&CK data into a Neo4j graph containing more than 300,000 nodes, enabling multi-hop traversal and topology-based link scoring. Usama et al. In [9] constructed separate threat-centric and asset-centric graphs and integrated them through entity alignment, with the resulting graphs validated against external gold standards. Other studies have focused on constructing knowledge graphs directly from unstructured CTI reports. Piplai et al. In [13] used CRF-based entity extraction together with a UCO-based schema that includes relations such as *mitigates*, while [1] combined RoBERTa-BiLSTM-CRF extraction with progressive entity linking and RotatE-based novelty detection. Wang et al. In [6] applied topic clustering and BERT-based semantic role labeling to extract subject–action–object triples from open-source threat reports. Although these approaches demonstrate the effectiveness of automated entity and relation extraction, their resulting graphs primarily represent the broader threat landscape, including actors, malware, and techniques, rather than relationships that directly characterize the risk of a vulnerability to a specific asset.

Some studies have moved toward a more asset-centric representation. Zou et al. In [5] jointly modeled network topology, system configuration, and vulnerability information and used the resulting graph for threat-path prediction and probability-based alerting. However, the approach does not explicitly define a multi-level risk classification based on the environmental context of an individual vulnerability. Li et al. In [11] constructed a vulnerability ontology linking vulnerabilities with products, vendors, risk levels, and categories and supported natural-language queries over the graph. While useful for vulnerability knowledge retrieval, the ontology has limited contextual depth and does not reason over operational conditions or threat-intelligence information.

Beyond graph construction, other studies have investigated graph refinement and reasoning to improve the quality and analytical capability of cybersecurity knowledge graphs. Shi et al. In [7] showed that pruning disconnected entities and merging version-specific entities before graph embedding can improve link-prediction performance on a CPE–CVE–CWE graph. Andrejeus et al. In [12] proposed a staged repair framework combining normalization, Markov-based entity-type smoothing, constrained LLM repair, and Skew Zero Forcing, recovering 30.9% of otherwise discarded schema-invalid CTI triples. Belhadj et al. In [10] combined graph neural networks with large language models for real-time graph validation and anomaly detection, although the evaluation was limited to synthetic microservice data. These studies improve graph completeness, consistency, or reasoning capability, but do not specifically address the use of graph-derived operational context for vulnerability risk prioritization.

Amin et al. In [2] is among the closest examples of an integrated classification and reporting pipeline. Their approach

combines honeypot-collected logs with an XGBoost classifier for attack-type detection, a Neo4j knowledge graph linking detected attack types to plausible CVEs, and a LoRA-fine-tuned LLaMA model for automated CTI report generation. While this demonstrates the feasibility of combining machine learning, knowledge graphs, and automated reporting, the classification focuses on four coarse-grained attack categories rather than vulnerability risk. Furthermore, the graph does not explicitly model asset criticality, operational exposure, or a multi-level risk outcome.

Overall, vulnerability classification and CTI knowledge graph construction have largely been investigated as separate problems. Existing classification approaches primarily infer vulnerability categories from textual descriptions, whereas knowledge graph approaches focus on representing and connecting cybersecurity entities. Even integrated pipelines such as [2] rely on general-purpose graph relationships rather than a schema designed around an environment-specific risk question. Similarly, asset-oriented approaches [5], [11] provide contextual representations but do not explicitly couple that context with a multi-level vulnerability risk assessment. These limitations motivate a Vulnerability Context Knowledge Graph (VCKG) designed to determine how the risk posed by a vulnerability changes according to the characteristics and operational context of the affected asset.

III. Our Proposed System

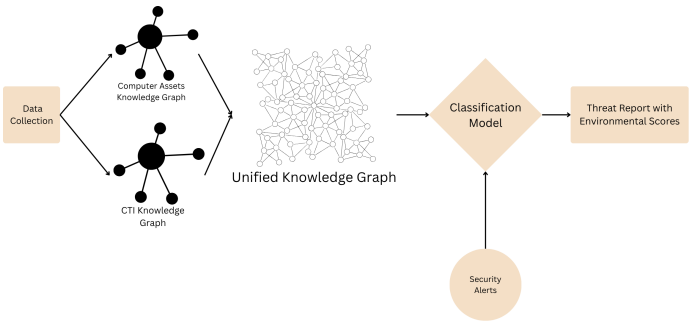


Fig. 1. Our proposed System Overview model

A. System Overview

Our system is designed to build on top of on the work of [1] to create a context-aware classification system using knowledge graph as its basis.

The proposed system consists of major parts as follows:

*1) **Unified Knowledge Graph Construction**:* We adopt the three-phase construction framework of Usama et al. [1] (data collection, knowledge graph construction, and graph integration through entity alignment and resolution) and adapt its integration phase to our setting. Based on the collected data, we construct two separate knowledge graphs: a CTI knowledge graph representing cybersecurity intelligence and a Computer Assets knowledge graph representing the laboratory network infrastructure. Each graph is constructed using its corresponding entity types and relationships. Whereas [1] aligned taxonomies through string similarity, our CTI sources share standardized identifiers, allowing deterministic alignment; the resolution effort is instead concentrated where [1] leaves the problem open, namely mapping an organization's deployed software to CPE identifiers.

- **Data Collection Phase** Two data streams are gathered for constructing the knowledge graphs: cyber threat intelligence (CTI) data from authoritative public sources and computer asset data from internal inventory systems. Data collection is performed through two mechanisms, depending on the availability and format of each source: (1) RESTful API requests, which retrieve structured data, typically in JSON format, from remote endpoints via HTTP requests; and (2) direct file downloads, which obtain complete datasets in XML or JSON format for subsequent local parsing and processing. For the CTI knowledge graph, we use structured data from authoritative CTI sources, including:
  - **MITRE CWE** A list of software and hardware weaknesses.
  - **MITRE CAPEC** A catalog and classification of attack patterns.
  - **MITRE ATT&CK** A knowledge base of adversary tactics and techniques based on real-world observations.
  - **CVE Program cvelist5** Records of publicly disclosed vulnerabilities and exposures.
  - **NVD CVE yearly feeds** Annual CVE data feeds from the National Vulnerability Database (NVD), used as a supplementary source to fill information gaps in cvelistV5.

  The CTI data are collected through source-specific methods. CWE and CAPEC are downloaded as XML files, while ATT/&CK data are retrieved in STIX 2.1 JSON format from GitHub. CVE records are obtained from the CVE List repository, and NVD data are retrieved through its REST API. The collected data are parsed into structured records for knowledge graph construction.
  For the computer assets graph, we collect asset information using both Wazuh and network inventory scanners. Wazuh Syscollector provides information about operating systems, network interfaces, installed software, and running services through the Wazuh API. Network inventory scanners are also used to discover and validate devices and services within the laboratory network. The collected information is processed into structured asset records for graph construction.
  The Confidentiality, Integrity, and Availability (CIA) requirements of each asset are manually assigned by the team and stored in a CSV file containing the asset hostname, operational role, CR, IR, AR, and justification. These values are subsequently imported into the corresponding Asset nodes.
- **Knowledge Graph Construction.** Each knowledge

graph is defined as a labeled, directed multi-graph:

$$G = (V, E, \lambda, \rho) \tag{1}$$

where V is the set of nodes, $E \subseteq V \times V$ is the set of directed edges, $\lambda : V \to L$ is a function that assigns each node a type label from the set of node types L, and $\rho : E \to R$ is a function that assigns each edge a relation type from the set of relation types R.

We first construct the CTI knowledge graph, which comprises nine node types and eleven relation types. CWE weaknesses serve as the central hub of each threat subgraph. Each weakness is connected to CVE vulnerabilities (`EXPLOITED_BY`), CAPEC attack patterns (`CAUSES_WEAKNESS`), ATT&CK techniques and tactics (`USES_TECHNIQUE` and `IMPLEMENTS`), threat groups, mitigations (`MITIGATED_BY`), detection methods (`DETECTED_BY`), and affected platforms (`AFFECTS`). The hierarchical structure of the threat taxonomies is preserved through the `IS_PARENT` and `SUBTECHNIQUE_OF` relations. Finally, CPE nodes are linked to their corresponding CVE vulnerabilities through `VULNERABLE_TO` relations. The resulting graph schema is illustrated in Fig. 2.

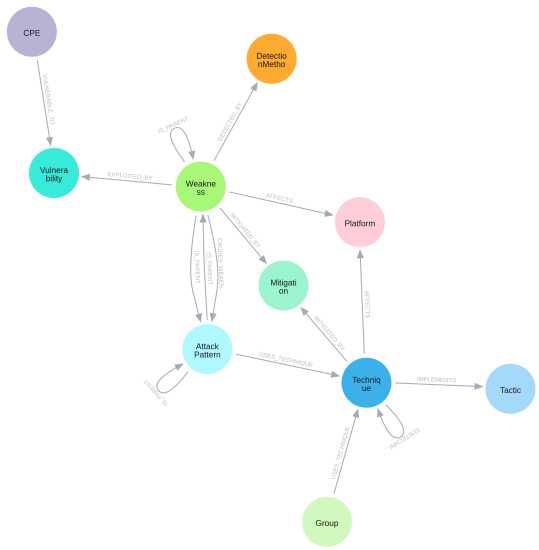


Fig. 2. CTI Knowledge Graph Schema

- **Computer Assets Graphs Construction**
  The computer asset graph $G_A = (V_A, E_A)$ represents the environment that vulnerability intelligence is applied to: which hosts exist, where they sit in the network, what software they run, how they communicate, and how important they are to the organization. Following the construction approach of Usama et al. [1], in which asset data are taken from the organization's own inventory, the graph is *domain-specific*: it describes one network and is rebuilt when that network changes. We build it for a laboratory network so that asset values and attack scenarios are controlled during evaluation. This section describes the data sources, the graph schema, and the construction procedure.

  The computer asset knowledge graph is designed to represent the network infrastructure and its associated security context through five node types and six relation types. Asset nodes form the core of the graph, representing hosts in the monitored network and storing key attributes, including hostname, IP address, operating system, role, and the CVSS v3.1 security requirements CR, IR, and AR. Each asset is associated with a network zone through the `IN_ZONE` relation, while communication between assets is represented by `CONNECTS_TO`. The `ALLOWED_TO_REACH` relation captures inter-zone communication permitted by firewall rules. At the service and software levels, the `RUNS` relation links assets to the services they host, and `IMPLEMENTED_BY` associates each service with its corresponding software product. Software products are further mapped to standardized Common Platform Enumeration (CPE) identifiers through the `IDENTIFIED_AS` relation. These CPE nodes are shared with the CTI knowledge graph, enabling the identification of vulnerabilities affecting software deployed on each asset through the `VULNERABLE_TO` relation. This integration establishes a link between network assets, their software dependencies, and known vulnerabilities, providing the contextual foundation for subsequent risk assessment. The resulting graph schema is illustrated.

  After the unified graph is constructed, security alerts from monitoring tools are used as input for graph-based analysis. Each alert is processed to identify the corresponding asset and vulnerability in the unified graph, allowing the system to determine the affected assets and their potential impact.

- **Graph Integration and Resolution** This is the final step of constructing the unified knowledge graph. In this step, we join the two KG through the shared common CPE nodes in the two graphs. The shared CPE identifiers provide a common reference for linking software deployed on computer assets to known vulnerabilities in the CTI knowledge graph. Through this identifier-based integration, the two graphs form a unified Vulnerability Context Knowledge Graph (VCKG), enabling queries across asset, software, and vulnerability relationships. Where the CPE nodes in CTI knowledge graph represents the platform CVE is found on and in CA knowledge graph it represents what platform the asset operates on. We link the two together with an identifier-based linking method.

$$G_{CTI} = (V_C, E_C, \lambda_C, \rho_C) \tag{2}$$

$$G_{CA} = (V_A, E_A, \lambda_A, \rho_A) \tag{3}$$

Where G_{CTI} and G_{CA} represent the CTI and CA knowledge graphs respectively. Given the knowledge graphs, an

alignment is a set $M \subseteq V_C \times V_A$ of pairs referring to the same real-world entity. The unified graph is

$$G_{\text{VKG}} = ((V_C \cup V_A)/\sim_M,\ E_C \cup E_A,\ \lambda,\ \rho) \quad (4)$$

where $\sim_M$ is the equivalence relation induced by M and aligned nodes are merged into a single node that keeps the union of their edges.

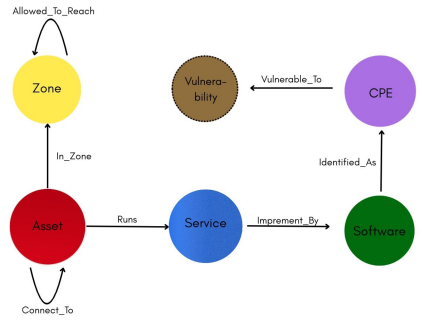


Fig. 3. Virtualization Graph of Computer Asset

*2) **Affected Assets Identification**:* In this phase of the system we search for the nearby assets nodes, in extension to the exploited nodes. In our laboratory environment, Suricata monitors network traffic and generates security alerts in EVE JSON format. These logs are collected by the Wazuh agent and forwarded to the Wazuh manager for centralized monitoring. The alert records are then retrieved and processed to extract relevant information, such as the destination IP address, destination port, and available vulnerability identifiers. When a monitoring tool raises an alert. it reports a vulnerability instance as a pair (a, c), where a $\in V_A$ is the affected asset and c is the reported CVE. The blast radius of this instance is the set of assets that are directly exposed as a consequence of c being exploitable on a. Rather than enumerating compute multi-step attack paths, we define the blast radius as a bounded reachability neighborhood over the asset layer of the unified graph G_{VKG}. This keep the computation deterministic and directly interpretable by analysts.

- **Asset Reachability Graph** The neighborhood of an asset in G_{VKG} contains nodes of heterogeneous types, including zones, services, and other assets. An unrestricted traversal would therefore return entities that do not represent exposed hosts. We first project G_{VKG} onto a directed Asset Reachability Graph $G_R = (V_R, E_R)$, where $V_R \subseteq V_A$ is the set of asset nodes and

$$E_R = \{(u,v) \in V_R \times V_R \mid (u,v) \in E_A,\ \rho(u,v) = \text{CONNECTS_TO},\ \pi(u,v)\} \quad (5)$$

The predicate $\pi(u, v)$ states that communication from u to v is permitted by the network policy:

$$\pi(u,v) \longleftrightarrow z(u) \vee \rho(z(u), z(v)) = \text{ALLOWED_TO_REACH} \quad (6)$$

Here $z(\cdot)$ maps an asset to its zone through the IN_ZONE relation. Zones therefore act as a constraint on traversal rather than as an expansion path. This prevents an entire subnet from being included merely because it shares a zone with the affected asset.

- **Impact-dependent Direction** The direction in which exposure propagates depends on the nature of the vulnerability. A vulnerability that compromises the confidentiality or integrity of a host allows an attacker to use that host as a pivot toward the assets it can reach. A vulnerability that affects only availability does not grant such a foothold, but it disrupts the assets that depend on the affected host. We derive the impact mode $m(c)$ from the CVSS v3.1 base vector of c. If the Confidentiality or Integrity impact metric is not $None$, $m(c) = compromise$. Otherwise, $m(c) = disruption$. The one-hop neighborhood of a is then

$$N(a,c) = N^+(a) = \{v : (a,v) \in E_R\} \quad (7)$$

if $m(c) = compromise$

$$N(a,c) = N^-(a) = \{u : (u,a) \in E_R\} \quad (8)$$

if $m(c) = disruption$

Blast Radius Generalizing to a traversal depth k, the blast radius of the vulnerability instance (a, c) is

$$BR_k(a,c) = \{v \in V_R : d_c(a,v) \leq k\} \quad (9)$$

where $d_c(a, v)$ is the shortest-path distance from a to v to G_R along the direction selected by $m(c)$. The affected asset itself is included as $BR_O(ac) = \{a\}$. In this work, we set $k = 1$, so the blast radius consists of the affected asset and its immediate neighbors under the reachability constraints of Eqs. (5)-(7). The procedure is given in Algorithm 1. It is a breadth-first search bounded at depth k and restricted to edges in E_R. For $k = 1$, its cost is $O(deg(a))$, which allows blast radius identification to run on every incoming alert.

- **Choice of depth** Restricting the traversal to s single hop reflects the purpose of the blast radius in our framework. Its role is to supply the environmental context needed to reassess the severity of the reported vulnerability, not to reconstruct full attack paths. Assets beyond the first hop can only be reached if the the attacker exploits further, independent weaknesses on intermediate hosts. Modeling those steps requires attack-graph generation, which is outside the scope of this work. The depth parameter k is retained in the formulation so the method can be extended to multi-hop analysis in future work.

The resulting set $BR_1(a, c)$, together with the CIA requirements (CR,IR,AR) of each asset it contains, forms

Algorithm 1 Blast Radius Identification

Require: G_{VKG}, alert (a, c), depth k
Ensure: $BR_k(a, c)$

```
m ← ImpactMode(CVSS(c))
BR ← {a};   frontier ← {a}
for i ← 1 to k do
    next ← ∅
    for each u in frontier do
        for each v in Neighbors(u, m) do
                                  ▷ edges in E_R per Eq. (7)
            if v ∉ BR then
                BR ← BR ∪ {v}
                next ← next ∪ {v}
            end if
        end for
    end for
    frontier ← next
end for
return BR
```

the input to the environmental scoring described in the next section.

*3) **Environmental Vulnerability Score**:* For a vulnerability c reported on an asset u, the Environmental Vulnerability Score $\mathrm{EVS}(c, u)$ is computed in two stages. First, the CVSS v3.1 Environmental equation is evaluated with its input metrics derived from the VKG rather than assigned manually. Second, a bounded propagation term accounts for the critical assets that an attacker could reach from u, which CVSS does not model.

- **CVSS v3.1 Environmental Score** Let $\lceil x \rceil_{0.1}$ denote the smallest number, to one decimal place, that is greater than or equal to x. The Modified Impact Sub-Score is

$$\mathrm{MISS} = \mathrm{MISS} = \min\Big(1 - (1 - CR \cdot MC)(1 - IR \cdot MI)(1 - AR \cdot MA),\ 0.915\Big) \tag{10}$$

where CR, IR, AR are the security requirements stored on the Asset node of u, and MC, MI, MA are the modified impact metrics of c. The modified impact and exploitability are

$$I_M = \begin{cases} 6.42 \cdot \mathrm{MISS}, & MS = U \\ 7.52(\mathrm{MISS} - 0.029) - \\ 3.25(0.9731\,\mathrm{MISS} - 0.02)^{13}, & MS = C \end{cases} \tag{11}$$

$$X_M = 8.22 \cdot MAV \cdot MAC \cdot MPR \cdot MUI \tag{12}$$

and the environmental score is

$$\mathrm{Env}(c, u) = \begin{cases} 0, I_M \le 0 \\ \Big\lceil \lceil \min(I_M + X_M,\ 10) \rceil_{0.1} \cdot T \Big\rceil_{0.1}, \\ MS = U \\ \Big\lceil \lceil \min(1.08(I_M + X_M),\ 10) \rceil_{0.1} \cdot T \Big\rceil_{0.1}, \\ MS = C \end{cases} \tag{13}$$

where $T = E \cdot RL \cdot RC$ is the temporal multiplier. Metric weights follow the CVSS v3.1 specification.

- **Graph-Derived Metric Assignment** Metrics not listed in Table I take their base values from the CVE node.

TABLE I
DERIVATION OF ENVIRONMENTAL METRICS FROM THE VKG

Metric	Derivation rule
CR, IR, AR	Attributes of the Asset node of u
MAV	Base AV if a path of `ALLOWED_TO_REACH` edges connects the Internet zone to the zone of u (via `IN_ZONE`); otherwise $AV{:}N$ is lowered to A
E	Exploitation Code Maturity
RL, RC	Not Defined

Exploit Code Maturity (E). CVSS v3.1 defines E as a temporal metric describing the current state of exploit techniques or code availability for a vulnerability [CVSS v3.1 specification]. Its values are Not Defined (X), High (H), Functional (F), Proof-of-Concept (P) and Unproven (U), with weights 1.0, 1.0, 0.97, 0.94 and 0.91. Because the standard leaves E to the assessor, we derive it from the evidence available in the unified graph, taking the strongest evidence first:

$$E(c, u) = \begin{cases} H & \text{if } \alpha(c, u) = 1; \vee; c \in \mathrm{KEV}; \vee; s(c) = \textit{active} \\ P & \text{if } s(c) = \textit{poc} \\ U & \text{if } s(c) = \textit{none} \\ X & \text{otherwise} \end{cases} \tag{14}$$

Here $\alpha(c, u) = 1$ if a security alert reports CVE c on asset u. KEV is the CISA Known Exploited Vulnerabilities catalogue. $s(c)$ is the Exploitation decision point of CISA's vulnrichment, which uses the values none, PoC and active from the SSVC guide.

- **Propagation Term** Let $R(u)$ be the set of assets reachable from u through `CONNECTS_TO` edges, where a hop between zones is permitted only if the zones are linked by `ALLOWED_TO_REACH`. Let $d(u, v)$ be the shortest-path hop distance. The criticality of an asset is its highest security requirement, normalized to $[0, 1]$:

$$\mathrm{crit}(v) = \max(CR_v, IR_v, AR_v) - 0.5 \tag{15}$$

The propagation score of u is

$$P(u) = \frac{1}{|V_A| - 1} \sum_{v \in R(u) \setminus \{u\}} \mathrm{crit}(v)\, \gamma^{\,d(u,v)} \tag{16}$$

where $\gamma \in (0,1)$ discounts distant assets. Since $\mathrm{crit}(v) \leq 1$ and $\gamma^d \leq 1$, $P(u) \in [0,1]$.

- **Final Score and Classification**

$$\mathrm{EVS}(c,u) = \min\Big(10,\ \mathrm{Env}(c,u) \cdot \big(1 + \beta\, P(u)\big)\Big) \quad (17)$$

where β bounds the maximum uplift. The risk level follows the CVSS v3.1 qualitative rating scale:

$$\ell(c,u) = \begin{cases} \text{Critical}, & 9.0 \leq \mathrm{EVS} \leq 10 \\ \text{High}, & 7.0 \leq \mathrm{EVS} < 9.0 \\ \text{Medium}, & 4.0 \leq \mathrm{EVS} < 7.0 \\ \text{Low}, & 0 < \mathrm{EVS} < 4.0 \end{cases} \quad (18)$$

Each graph signal contributes to exactly one term: Internet exposure affects only MAV, and lateral reachability affects only $P(u)$, so no context is counted twice.

*4) **Context-Aware Threat Report**:* After the EVS calculation and risk classification, the system collects the assessment results to prepare the input for report generation. These results include the original alert information, affected asset details, CVE identifier, base CVSS score, calculated EVS, assigned risk level, affected assets, and corresponding mitigation information retrieved from the VCKG. The collected information is organized into a structured JSON object.

The structured assessment data are then provided to a Large Language Model (LLM) to generate a human-readable threat report. The LLM is instructed to summarize the security event, describe the affected assets, explain the environmental risk assessment, and present relevant mitigation recommendations. Importantly, the LLM is responsible only for interpreting and presenting the assessment results; vulnerability scoring and risk classification are performed by the proposed system.

To maintain consistency with the calculated results, the generated report is validated against the original structured input. Key information, including CVE identifiers and numerical assessment values, is checked to identify inconsistencies or unsupported information before the report is finalized.

REFERENCES

[1] M. Usama, K. Nagaty and N. Elmaghawry, "Construction of a Unified Knowledge Graph for Cyber Threat Intelligence," 2025 International Conference on Computer and Applications (ICCA), Bahrain, Bahrain, 2025, pp. 1-9

[2] Yukun Dong, Yeer Tang, Xiaotong Cheng, Yufei Yang, DeKeDVer: A deep learning-based multi-type software vulnerability classification framework using vulnerability description and source code, Information and Software Technology, Volume 163, 2023, 107290, ISSN 0950-5849,

[3] X. Liu, Y. Tian, L. Tang and S. Zeng, "A Framework for Automated Cyber Threat Intelligence Extraction Based on Knowledge Graph," 2026 38th Chinese Control and Decision Conference (CCDC), Nanjing, China, 2026, pp. 4655-4660

[4] M. Rahman et al., "ATT&CK to CVE: A Large-Scale Automated Knowledge Graph for Threat Intelligence," 2025 IEEE International Conference on Data Mining Workshops (ICDMW), Washington, DC, USA, 2025, pp. 1044-1054

[5] A. Piplai, S. Mittal, A. Joshi, T. Finin, J. Holt and R. Zak, "Creating Cybersecurity Knowledge Graphs From Malware After Action Reports," in IEEE Access, vol. 8, pp. 211691-211703, 2020

[6] S. S. Das, E. Serra, M. Halappanavar, A. Pothen and E. Al-Shaer, "V2W-BERT: A Framework for Effective Hierarchical Multiclass Classification of Software Vulnerabilities," 2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA), Porto, Portugal, 2021, pp. 1-12

[7] W. Wang, X. Han, Q. Zhao, L. Han, G. Wei and D. Xu, "Threat intelligence analysis based on multi-dimensional feature topic clustering and knowledge graph," 2023 5th International Conference on Frontiers Technology of Information and Computer (ICFTIC), Qiangdao, China, 2023, pp. 297-301

[8] C. Ma, D. Zhang, J. Zhang, L. Wang, H. Li and Z. Gu, "Cybersecurity Knowledge Graph Enables Targeted Data Collection for Cybersecurity Analysis," 2023 8th International Conference on Data Science in Cyberspace (DSC), Hefei, China, 2023, pp. 583-590

[9] Y. Li, Y. Guo, Y. Hao, Y. Wang, A. Yin and Y. Liu, "Intelligent Answer System Based on Vulnerability Knowledge Graph," 2021 7th International Conference on Computer and Communications (ICCC), Chengdu, China, 2021, pp. 1646-1651

[10] Y. Yang and Y. Zhao, "Network Security Threat Intelligence Modeling Based on Knowledge Graph," 2024 5th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+AI), Jingdezhen, China, 2024, pp. 154-158

[11] Z. Shi, N. Matyunin, K. Graffi and D. Starobinski, "Uncovering Product Vulnerabilities with Threat Knowledge Graphs," 2022 IEEE Secure Development Conference (SecDev), Atlanta, GA, USA, 2022, pp. 84-90

[12] TTMFN: Two-stream Transformer-based Multimodal Fusion Network for Survival Prediction, arXiv, 2023.

[13] Security model utilizing multi-channel data with risk-entity facing cybersecurity alert engine and portal (CVSS/NIST-style threshold definition).

[14] Castro-García, N., Muñoz Castañeda, Á. L., & Fernández-Rodríguez, M. (2020). Machine learning for automatic assignment of the severity of cybersecurity events. Computational and Mathematical Methods, 2, e1072.